UNIDAD 1

Fundamentos y errores en el cálculo numérico.

TEMA 1:

Sistemas numéricos y representación en punto flotante.

Descripción:

La asignatura de Métodos Numéricos proveerá al alumno de herramientas que permiten obtener soluciones aproximadas a problemas de ingeniería que no poseen una solución analítica exacta.

Para emplear estas herramientas adecuadamente, resulta necesario comprender los principios fundamentales, sobre los cuales se sustenta la representación numérica en las computadoras y sus limitaciones.

Esta unidad proporciona los cimientos preliminares de la asignatura y empieza con una revisión de la representación de los sistemas numéricos decimal, binarios y estándar de punto flotante.

Métodos numéricos y problemas complejos.

PERIODO ACADÉMICO 2026-1S

TABLA DE CONTENIDO

Tema 1. Sistemas numéricos y representación en punto flotante..... 2
1.1. Sistemas numéricos..... 2
1.2. Conversión entre sistemas..... 4
1.3. Representación de números enteros. 13
1.4. Representación de números reales mediante punto flotante. 15
Bibliografía 19

Tema 1. Sistemas numéricos y representación en punto flotante.

1.1. Sistemas numéricos.

Para empezar el estudio de los métodos numéricos, es necesario diferenciar dos conceptos que se encuentran íntimamente ligados y sobre los cuales se sustenta esta diciplina: **el problema numérico y el algoritmo.** Un **problema numérico** se concibe como la formulación precisa y sin ambigüedades de una situación matemática y establece de manera formal la relación funcional que existe entre un conjunto de datos de entrada (las variables independientes) y sus correspondientes datos de salida (las variables dependientes). En otras palabras, el **problema numérico** define con total claridad **qué es lo que se busca resolver**, sin que todavía se defina un algoritmo a seguir. (Escalante, 2022).

El algoritmo se refiere a **cómo** se va a resolver el problema numérico. Se trata de una secuencia finita y ordenada de operaciones lógicas y matemáticas bien definidas, que detallan el procedimiento exacto para transformar un conjunto de datos de entrada en datos de salida. Cabe destacar que un único problema numérico no está atado a un único algoritmo. Por el contrario, puede ser abordado mediante diferentes algoritmos. **La elección del algoritmo es una decisión de ingeniería**, ya que cada uno posee un nivel de precisión diferente y, a menudo, un costo computacional distinto. Las soluciones obtenidas por los algoritmos son **aproximaciones**, por lo cual el término **"error"** estará bastante presente en la asignatura (Escalante, 2022). El alumno debe desarrollar los criterios que le permitan seleccionar el algoritmo más adecuado para obtener una solución con la exactitud requerida y de la manera más eficiente posible.

1.1.1. Sistema numérico decimal.

El sistema decimal es la forma convencional en la cual entendemos el mundo y se basa en **10** dígitos (0, 1, 2, 3, 4, 5, 6, 7, 8, 9). En el **sistema decimal,** un número real está compuesto por un signo, una **parte entera** y una **parte decimal** (Altaç, 2025).

Resulta conveniente repasar la estructura de un número, tomando como ejemplo el número 430.295. Este número está escrito en el **sistema decimal**, y como se puede apreciar, está conformado por 6 dígitos (4, 3, 0, 2, 9, 5) y un signo decimal. La parte entera es 430 y la decimal 295, estos se hallan separados por el signo o separador decimal. Como signo decimal se utiliza el punto o la coma, para efectos de este curso **se utilizará el punto**.

Tabla 1. Números en sistema decimal.

Parte entera				Parte decimal		
4	**3**	**0**	**.**	**2**	**9**	**5**
Centenas	Decenas	Unidades	Signo decimal	Décimas	Centésimas	Milésimas

Fuente: (Tussy, Gustafson, & Koenig, 2013)

1.1.2. Sistema Binario.

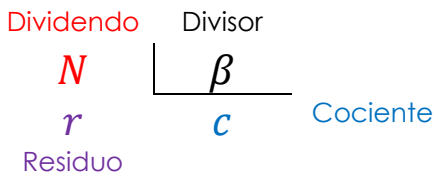
La computación digital se basa en el **sistema binario**, un lenguaje elemental basado en dos únicos estados: la presencia (**1**) o ausencia (**0**) de un pulso eléctrico, es decir, emplea dos dígitos. A esta unidad mínima de información se denomina **bit** (Altaç, 2025). El número 9 que conocemos del sistema decimal, en el sistema binario se escribiría como: 1001. Es decir $(9)_{10} = (1001)_2$

Para gestionar datos complejos, los bits se **agrupan en bytes (conjunto de 8 bits) y en palabras**, debido a que así se optimiza el procesamiento. Aunque el binario es el lenguaje nativo de las máquinas, existen otros sistemas como el octal y hexadecimal, permitiendo a los programadores y analistas interpretar y manipular grandes volúmenes de datos de forma más compacta y eficiente (Altaç, 2025).

1.2. Conversión entre sistemas.

1.2.1. De sistema decimal a binario (parte entera N).

Para transformar un número entero N, del sistema decimal a un sistema con una base diferente β, es necesario recordar los principios básicos del algoritmo de la división (González, Chacón, & Fonseca, 2023).



De la división podemos concluir que:
Dividendo=Divisor x Cociente + Residuo
Es decir:

$$\boldsymbol{N = (\beta \times c) + r}$$

Por lo tanto, se sigue el algoritmo de divisiones sucesivas, que se muestra a continuación:

1. Se toma el número en base decimal N original y se divide entre la nueva base β. De esta operación se anota el cociente c_1 y el residuo r_1.
2. Se realizan divisiones iterativas, es decir, se toma el cociente c_1y se vuelve a dividir por la base β. Se anota el nuevo cociente c_2 y el residuo r_2.
3. El proceso se repite hasta obtener un cociente igual a cero y un residuo r_i.
4. Se organiza una tabla en base a la conclusión obtenida:

$$N = (c \times \beta) + r$$

El número equivalente se forma de la siguiente manera $(r_i r_{i-1} r_{i-2} \dots r_1)_\beta$. Es decir, se agrupa los residuos **desde abajo hacia arriba** de la tabla (Nieves, 2015).

Se puede **resumir en una tabla**, como se muestra a continuación:

Tabla 2. Conversión de sistema decimal a binario.

Dividendo		Base β		Cociente		Residuo
$\boldsymbol{R}$	=	β	×	c_1	+	r_1
$\boldsymbol{c_1}$	=	β	×	c_2	+	r_2
$\boldsymbol{c_2}$	=	β	×	c_3	+	r_3
⋮		⋮		⋮		⋮
$\boldsymbol{c_{i-1}}$	=	β	×	c_i	+	r_i

Por lo que $(R)_{10} = (r_i r_{i-1} r_{i-2} \dots r_1)_\beta$

- **Ejemplo:**

Convertir el número 30 decimal a binario.

Dividendo		Base β (divisor)		Cociente		Residuo
30	=	**2**	×	**15**	+	**0**
15	=	**2**	×	**7**	+	**1**
7	=	**2**	×	**3**	+	**1**
3	=	**2**	×	**1**	+	**1**
1	=	**2**	×	**0**	+	**1**

Por lo tanto: $(30)_{10} = (\mathbf{11110})_2$

Ejemplo:

Convertir el número 91 decimal a binario.

Dividendo		Base β (divisor)		Cociente		Residuo
91	=	2	×	45	+	1
45	=	2	×	22	+	1
22	=	2	×	11	+	0
11	=	2	×	5	+	1
5	=	2	×	2	+	1
2	=	2	×	1	+	0
1	=	2	×	0	+	1

Por lo tanto: $(91)_{10} = (1011011)_2$

1.2.2. De sistema decimal a binario (parte decimal).

En este caso, se multiplica el número en sistema decimal por la base binaria 2. El producto obtenido, se separa en una parte fraccionaria y una entera (0 y 1). El número binario, se forma tomando los dígitos binarios de arriba hacia abajo.

Ejemplo:

Convertir 0.25 a binario.

Número base 10		Base β (binaria)		Producto		Parte fraccionaria		Parte Entera
0.25	×	2	=	0.5	=	0.5	+	0
0.5	×	2	=	1	=	0	+	1

Por lo tanto: $(0.25)_{10} = (0.01)_2$

Ejemplo:

Convertir 0.75 a binario.

Número base 10		Base β (binaria)		Producto		Parte fraccionaria		Parte Entera
0.75	$\times$	2	$=$	1.5	$=$	0.5	$+$	1
0.5	$\times$	2	$=$	1	$=$	0	$+$	1

Por lo tanto: $(0.75)_{10} = (0.11)_2$

Ejemplo:

Convertir 0.2 a binario.

Número base 10		Base β (binaria)		Producto		Parte fraccionaria		Parte Entera
0.2	$\times$	2	$=$	0.4	$=$	0.4	$+$	0
0.4	$\times$	2	$=$	0.8	$=$	0.8	$+$	0
0.8	$\times$	2	$=$	1.6	$=$	0.6	$+$	1
0.6	$\times$	2	$=$	1.2	$=$	0.2	$+$	1
0.2	$\times$	2	$=$	0.4	$=$	0.4	$+$	0
0.4	$\times$	2	$=$	0.8	$=$	0.8	$+$	0
0.8	$\times$	2	$=$	1.6	$=$	0.6	$+$	1
0.6	$\times$	2	$=$	1.2	$=$	0.2	$+$	1

Como se puede observar los dígitos binarios 0011 se repiten y lo hará infinitamente, es decir, $(0.2)_{10} = (0.00110011001100110011\ldots)_2$. Para representar a los dígitos que se **repiten infinitamente**, se acostumbra a utilizar una **barra** sobre ellos $(0.00110011001100110011\ldots)_2 = (0.\overline{0011})_2$. Por lo tanto: $(0.2)_{10} = (0.\overline{0011})_2$

Ejemplo:

Convertir 30.2 a binario.

Para convertir este número a binario, se lo debe hacer en dos partes. La primera corresponde a la parte entera y la segunda a la fraccionaria.

Para la parte **entera: 30**

Dividendo		Base β (divisor)		Cociente		Parte Entera
30	=	2	×	15	+	0
15	=	2	×	7	+	1
7	=	2	×	3	+	1
3	=	2	×	1	+	1
1	=	2	×	0	+	1

$$(30)_{10} = (11110)_2$$

Para la parte **fraccionaria 0.2**:

Número base 10		Base β (binaria)		Producto		Parte fraccionaria		Parte Entera
0.2	×	2	=	0.4	=	0.4	+	0
0.4	×	2	=	0.8	=	0.8	+	0
0.8	×	2	=	1.6	=	0.6	+	1
0.6	×	2	=	1.2	=	0.2	+	1
0.2	×	2	=	0.4	=	0.4	+	0
0.4	×	2	=	0.8	=	0.8	+	0
0.8	×	2	=	1.6	=	0.6	+	1
0.6	×	2	=	1.2	=	0.2	+	1

$$(0.2)_{10} = (0.\overline{0011})_2$$

Por lo tanto, al combinar los resultados tenemos que:

$$(30.2)_{10} = (11110.\overline{0011})_2$$

1.2.3. De sistema binario a decimal.

Como se mencionó en la parte inicial, la representación decimal de un número N, en base de un número β, se puede expresar como:

$$N = \sum_{k=0}^{n} a_k \times \beta^k$$

Por lo tanto, para la representación decimal de un número N en binario, es decir, en base 2, se tiene:

$$N = \sum_{k=0}^{n} a_k \times 2^k$$

Ejemplo:

Convertir el número binario 1100 a sistema decimal.

3	2	1	0	← Enumeración Desde 0 hasta n.
1	1	0	0	← Dígitos (0 y 1)
a_3	a_2	a_1	a_0	

$$N = \sum_{k=0}^{n} a_k \times 2^k$$

$$N = a_n \times 2^n + a_{n-1} \times 2^{n-1} + \cdots + a_1 \times 2^1 + a_0 \times 2^0$$

$$N = a_3 \times 2^3 + a_2 \times 2^{3-1} + a_1 \times 2^1 + a_0 \times 2^0$$

$$N = \mathbf{1} \times 2^3 + \mathbf{1} \times 2^2 + \mathbf{0} \times 2^1 + \mathbf{0} \times 2^0$$

$$N = \mathbf{1} \times 8 + \mathbf{1} \times 4 + \mathbf{0} \times 2 + \mathbf{0} \times 1$$

$$N = 8 + 4 + 0 + 0$$

$$N = 12$$

$$(\mathbf{1100})_{\mathbf{2}} = (\mathbf{12})_{\mathbf{10}}$$

- **Ejemplo:**

Convertir el número binario 100101 a sistema decimal.

5	4	3	2	1	0	← Enumeración Desde 0 hasta n.
1	0	0	1	0	1	← Dígitos (0 y 1)
a_5	a_4	a_3	a_2	a_1	a_0	

$$N = \sum_{k=0}^{n} a_k \times 2^k$$

$$N = a_n \times 2^n + a_{n-1} \times 2^{n-1} + \cdots + a_1 \times 2^1 + a_0 \times 2^0$$

$$N = a_5 \times 2^n + a_4 \times 2^{n-1} + a_3 \times 2^{n-2} + a_2 \times 2^{n-3} + a_1 \times 2^{n-4} + a_0 \times 2^{n-5}$$

$$N = \mathbf{1} \times 2^5 + \mathbf{0} \times 2^4 + \mathbf{0} \times 2^3 + \mathbf{1} \times 2^2 + \mathbf{0} \times 2^1 + \mathbf{1} \times 2^0$$

$$N = \mathbf{1} \times 32 + \mathbf{0} \times 16 + \mathbf{0} \times 8 + \mathbf{1} \times 4 + \mathbf{0} \times 2 + \mathbf{1} \times 1$$

$$N = 32 + 0 + 0 + 4 + 0 + 1$$

$$N = 37$$

$$(\mathbf{100101})_{\mathbf{2}} = (\mathbf{37})_{\mathbf{10}}$$

- **Ejemplo:** convertir el número binario 0.11 a sistema decimal.

$$F = \sum_{k=1}^{\infty} b_k \times 2^{-k}$$

$$F = b_1 \times 2^{-1} + b_2 \times 2^{-2} + \cdots + b_n \times 2^{-n} + \cdots$$

	1	2	← Enumeración Desde 1 hasta n.
0	.1	1	← Dígitos (0 y 1)

	b_1	b_2	

$$F = (1 \times 2^{-1}) + (1 \times 2^{-2})$$

$$F = \left(1 \times \frac{1}{2}\right) + \left(1 \times \frac{1}{2^2}\right) = \frac{1}{2} + \frac{1}{4} = 0.5 + 0.25$$

$$F = 0.75$$

$$(\mathbf{0.11})_{\mathbf{2}} = (\mathbf{0.75})_{\mathbf{10}}$$

- **Ejemplo:** convertir el número binario 0.0101 a sistema decimal.

$$F = \sum_{k=1}^{\infty} b_k \times 2^{-k}$$

$$F = b_1 \times 2^{-1} + b_2 \times 2^{-2} + \cdots + b_n \times 2^{-n} + \cdots$$

	1	2	3	4	← Enumeración Desde 1 hasta n.
0	.0	1	0	1	← Dígitos (0 y 1)
	b_1	b_2	b_3	b_4	

$$F = (\mathbf{0} \times 2^{-1}) + (\mathbf{1} \times 2^{-2}) + (\mathbf{0} \times 2^{-3}) + (\mathbf{1} \times 2^{-4})$$

$$F = \left(\mathbf{0} \times \frac{1}{2}\right) + \left(\mathbf{1} \times \frac{1}{2^2}\right) + \left(\mathbf{0} \times \frac{1}{2^3}\right) + \left(\mathbf{1} \times \frac{1}{2^4}\right) = 0 + \frac{1}{4} + 0 + \frac{1}{16}$$

$$F = 0 + 0.25 + 0 + 0.0625 = 0.3125$$

$$(\mathbf{0.0101})_{\mathbf{2}} = (\mathbf{0.3125})_{\mathbf{10}}$$

- **Ejemplo:** convertir el número binario 10100.11 a sistema decimal.

Se toma la **parte entera**: 10100

4	3	2	1	0	← Enumeración Desde 0 hasta n.
1	**0**	**1**	**0**	**0**	← Dígitos (0 y 1)
a_4	a_3	a_2	a_1	a_0	

$$N = \sum_{k=0}^{n} a_k \times 2^k$$

$$N = a_n \times 2^n + a_{n-1} \times 2^{n-1} + \cdots + a_1 \times 2^1 + a_0 \times 2^0$$

$$N = a_4 \times 2^{n-1} + a_3 \times 2^{n-2} + a_2 \times 2^{n-3} + a_1 \times 2^{n-4} + a_0 \times 2^{n-5}$$

$$N = (\mathbf{1} \times 2^4) + (\mathbf{0} \times 2^3) + (\mathbf{1} \times 2^2) + (\mathbf{0} \times 2^1) + (\mathbf{0} \times 2^0)$$

$$N = (\mathbf{1} \times 16) + (\mathbf{0} \times 8) + (\mathbf{1} \times 4) + (\mathbf{0} \times 2) + (\mathbf{0} \times 1)$$

$$N = 16 + 0 + 4 + 0 + 0$$

$$N = 20$$

$$(\mathbf{10100})_{\mathbf{2}} = (\mathbf{20})_{\mathbf{10}}$$

Se toma la **parte decimal**: 0.11

$$F = \sum_{k=1}^{\infty} b_k \times 2^{-k}$$

$$F = b_1 \times 2^{-1} + b_2 \times 2^{-2} + \cdots + b_n \times 2^{-n} + \cdots$$

	1	2	← Enumeración Desde 1 hasta n.
0	**.1**	**1**	← Dígitos (0 y 1)
	b_1	b_2	

$$F = (1 \times 2^{-1}) + (1 \times 2^{-2})$$

$$F = \left(1 \times \frac{1}{2}\right) + \left(1 \times \frac{1}{2^2}\right) = \frac{1}{2} + \frac{1}{4} = 0.5 + 0.25$$

$$F = 0.75$$

$$(\mathbf{0.11})_{\mathbf{2}} = (\mathbf{0.75})_{\mathbf{10}}$$

Por lo tanto: $(\mathbf{10100.11})_{\mathbf{2}} = (\mathbf{20.75})_{\mathbf{10}}$

1.2.4. Representación numérica en el computador.

En las computadoras, los números se representan usando una **cantidad limitada de bits**. Esta cantidad o "longitud" puede variar según el tipo de máquina. Por ejemplo, en cálculos científicos y de ingeniería, se requiere una mayor precisión, por lo que se utilizan longitudes de bits más grandes. Para tareas de procesamiento administrativo o cálculos menos complejos, es más práctico y eficiente trabajar con longitudes más cortas (Nieves, 2015).

En informática, a esta **cadena de bits se le denomina palabra**. El tamaño de una palabra puede variar desde 8 hasta 64 bits. Para facilitar su manejo, estas palabras se dividen en segmentos más pequeños llamados bytes. Como se mencionó al principio, **un byte** está conformado por **8 bits**, por lo que, una palabra de 32 bits se compondría por 4 bytes (Nieves, 2015).

1.3. Representación de números enteros.

1.3.1. Método de magnitud con signo.

El enfoque más **sencillo** (no necesariamente más eficiente a nivel de hardware) que permite comprender cómo se almacenan los números con signo, se conoce como el **método de magnitud con signo**. En este sistema, al bit inicial se le denomina bit más significativo o MSB y sirve para

representar un signo. **Un 0 indica que el valor es positivo, mientras que un 1 señala que es negativo**. Los bits restantes se utilizan para almacenar la magnitud del número (Chapra & Canale, 2021).

- **Ejemplo:**

  Supongamos que se requiere almacenar el número -173 en la memoria de una computadora de 16 bits.

Dividendo		Base β (divisor)		Cociente		Residuo
173	=	2	×	86	+	1
86	=	2	×	43	+	0
43	=	2	×	21	+	1
21	=	2	×	10	+	1
10	=	2	×	5	+	0
5	=	2	×	2	+	1
2	=	2	×	1	+	0
1	=	2	×	0	+	1

$$(173)_{10} = (10101101)_2$$

Mediante el sistema **de magnitud con signo** el número -173 se almacenaría de esta manera:

16 bits															
1	0	0	0	0	0	0	0	1	0	1	0	1	1	0	1
Signo	Número														

El tipo de dato entero, conocido como int, en el sistema de complemento a 2 se caracteriza por usar un espacio fijo de 32 bits en memoria, independientemente del valor numérico que almacene. Esta asignación de tamaño fijo establece una limitación al momento de almacenar números demasiado grandes (Gregori & Roig, 2019).

$$Valor\ máximo = valor(01111111111111111111111111111111) = 2^{31} - 1 = 2147483647$$

$$Valor\ mínimo = valor(10000000000000000000000000000000) = -2^{31} = -2147483648$$

En la siguiente tabla se muestran los tipos de enteros habituales y sus rangos de representación.

Tabla 3. Tipos de representación habitual de números enteros, en complemento a 2

Tipo	Bits	Rango
int (Entero)	32	$[-2^{31}, 2^{31} - 1]$
short (Entero corto)	16	$[-2^{15}, 2^{15} - 1]$
long (Entero largo)	64	$[-2^{63}, 2^{63} - 1]$

Fuente: (Gregori & Roig, 2019)

1.4. Representación de números reales mediante punto flotante.

1.4.1. Punto flotante.

Además de los números enteros, ya sean con o sin signo, los lenguajes de **programación manejan números fraccionarios**, conocidos en **matemáticas como números reales**. (Patterson & Hennessy, 2021)

$$3.14159265\ldots_{10}\ (\text{pi}) - (Recuerde\ que\ tiene\ infinitos\ decimales)$$

$$2.71828\ldots_{10}\ (e) - (Recuerde\ que\ tiene\ infinitos\ decimales)$$

$$0.000000001_{10}\ \text{o también}\ 1.0_{10} \times 10^{-9}$$

$$3\ 155\ 760\ 000_{10}\ o\ también\ 3.15576_{10} \times 10^{9}$$

En los dos últimos casos se tienen cantidades muy pequeñas y muy grandes. Por lo que, en el sistema decimal es bastante típico optar por el uso de la notación científica para representarlos. Ya que el número de

bits limita los valores que se pueden almacenar en un ordenador, una de las formas de ampliar el rango de representación es utilizar lo que se denomina **punto flotante** (Patterson & Hennessy, 2021).

La representación en punto flotante constituye el estándar más utilizado para expresar números reales dentro de los computadores. A diferencia de la representación entera o de punto fijo, el punto flotante permite manejar un rango mucho más amplio de valores. Esta representación de se basa en la notación científica, donde un número se expresa como un producto de un **coeficiente (mantisa o fracción)** y una potencia de una base. En la informática, esta base es generalmente 2 (Patterson & Hennessy, 2021).

$$(-1)^S \times M \times \beta^E \qquad donde: \beta^{-1} \leq M < 1$$

Un número en punto flotante se descompone en tres partes principales:

1. **Bit de signo (S):** Indica si el número es positivo (0) o negativo (1).
2. **Exponente (E):** Determina el orden de magnitud del número. El exponente permite que el punto decimal "flote".
3. **Mantisa (M):** Contiene los dígitos significativos del número $(0.d_1d_2d_3 \ldots)$ y representa la precisión.

Nota: La base β puede tomar un valor de 2 para el sistema binario o de 10 para el decimal.

- **Ejemplo:**

Para el sistema decimal: $\mathbf{-4358.97 = -4.35897 \times 10^3}$	El punto "flotó" hacia la izquierda, tres posiciones.
Para el sistema binario: $\mathbf{(1)\ 1011.01 = (1)\ 1.01101 \times 2^3}$	El punto "flotó" hacia la izquierda, tres posiciones. El (1) representa al signo menos.

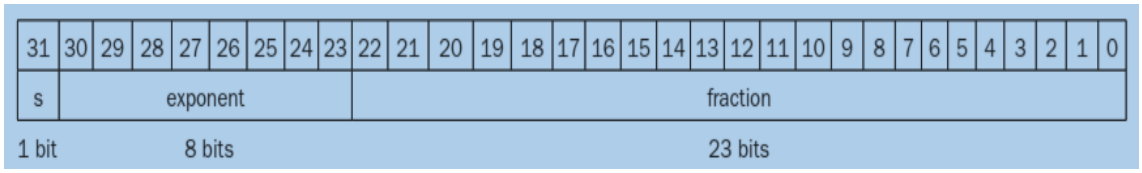
1.4.2. Estándar IEEE 754-2008 para punto flotante.

El Instituto para Ingenieros Eléctricos y Electrónicos publicó el estándar para la aritmética binaria de punto flotante donde se proporciona estándares para números de punto flotante decimales y binarios, formato de intercambio de datos etc. (Burden, Faires, & Burden, 2017).

El estándar de representación en punto flotante según la IEEE 754 para **precisión simple utiliza 32 bits** (4 bytes). El bit más significante corresponde al signo S, 8 bits al exponente E y 23 bits a la mantisa M (o parte fraccionaria). (Moreno C. , 2021)

Figura 1.

Asignación de bits para precisión simple (Patterson & Hennessy, 2021).



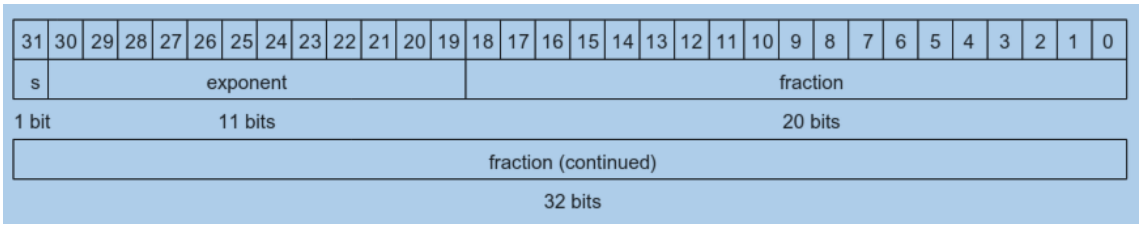
Fuente:(Patterson & Hennessy, 2021).

Con el uso de esta representación de exponente y mantisa (fracción) le dan a la aritmética de computadora un rango extraordinario.

Se pueden representar valores mucho más pequeños y valores mucho más grandes. Para incrementar aún más el rango se hace uso del número flotante de **doble precisión utilizando dos palabras de 32 bits (64 bits)**, es decir, se utiliza el doble de bits que se utilizan para un entero. En tal caso 1 bit corresponde al signo S, 11 al exponente E y 52 a la mantisa M o parte fraccionaria (Gregori & Roig, 2019).

Figura 2.

Asignación de bits para precisión doble (Patterson & Hennessy, 2021).



Fuente:(Patterson & Hennessy, 2021).

Como ya se puede intuir, el almacenamiento de un número real, está limitado por el número de bits. Es decir que, con un número limitado de bits, no se puede representar la infinita cantidad de números reales, por lo cual se estará expuesto a casos de desbordamiento (overflow) o de "subdesbordamiento" (underflow) (Patterson & Hennessy, 2021).

Desbordamiento: Se refiere a que el exponente es demasiado grande para ser representado en el campo del exponente.

Subdesbordamiento: Esta situación ocurre cuando el exponente negativo es menor al mínimo representable (más negativo), para caber en el campo del exponente.

Cualquiera de estos eventos podría dar como resultado que un programa dé **respuestas incorrectas**. (Patterson & Hennessy, 2021).

Bibliografía

Agudelo, N. (2021). *Métodos numéricos con Excel: guía práctica para ingenieros.* Publicaciones Universidad de América. Obtenido de https://elibro.net/es/ereader/unachecuador/223226

Altaç, Z. (2025). *Numerical Methods for Scientists and Engineers.* CCR Press.

Burden, R., Faires, D., & Burden, A. (2017). *Análisis Numérico* (10a ed.). Cengage Learning.

Chapra, S., & Canale, R. (2021). *Numerical Methods for Engineers* (8th ed.). New York: McGraw-Hill Education.

Escalante, R. (2022). *Métodos numéricos: una introducción a las matemáticas del cálculo científico.* Universidad de Alcalá. Obtenido de https://elibro.net/es/ereader/unachecuador/227876?

González, J., Chacón, B., & Fonseca, L. (2023). *Matemática básica.* UNIMAR.

González, J., Chacón, J., & Fonseca, L. (2023). *Matemática básica.* Universidad Mariana. Obtenido de https://elibro.net/es/ereader/unachecuador/280647

Gregori, V., & Roig, B. (2019). *Errores, optimización y resolución numérica de sistemas.* Valencia: Editorial de la Universidad Politécnica de Valencia. Obtenido de https://elibro.net/es/ereader/unachecuador/111493

Moreno, C. (2021). *Introducción al cálculo numérico.* Universidad Nacional de Educación a Distancia. Obtenido de https://elibro.net/es/ereader/unachecuador/213209

Nieves, A. (2015). *Métodos numéricos: aplicados a la ingeniería.* Grupo Editorial Patria. Obtenido de https://elibro.net/es/ereader/unachecuador/39455

Patterson, D., & Hennessy, J. (2021). *Computer Organization and Design.* Kidlington: Morgan Kaufmann.

Tussy, A., Gustafson, R., & Koenig, D. (2013). *Matemáticas Básicas.* CENGAGE Learning.